

Aliona Pirozhenko

In this assignment, I implemented a simple IPv4 router inside a controlled Mininet environment. The goal was to correctly handle Ethernet, ARP, IP, and ICMP protocols and to forward packets according to a routing table while following specific protocol behaviors. The implementation required completing three functions: `sr_handlepacket`, `ip_black_list`, and `sr_arpcache_handle_arpreq`. After implementation, the router was tested using tools such as `ping` and `traceroute` to verify correctness.

[illegible]

sr_handlepacket Implementation

The `sr_handlepacket` function is the core of the router logic. It is invoked whenever an Ethernet frame arrives on any interface, and it determines how the packet should be processed. I began by validating the packet length and parsing the Ethernet header. Based on the Ethernet type, the function distinguishes between IP packets and ARP packets. Any other Ethernet types are ignored as required. For IP packets, I first verified that the packet was IPv4 and that the IP checksum was correct. Invalid packets were dropped immediately. I then applied the firewall logic by invoking `ip_black_list` to block packets whose source or destination IP matched the given blacklist subnet. Next, I checked whether the destination IP address matched one of the router's own interfaces. If so, the packet was treated as destined to the router itself. In this case:

- ICMP echo requests were answered with ICMP echo replies.
- TCP or UDP packets were rejected by sending an ICMP port unreachable message.
- Other protocols were ignored.

If the packet was not destined to the router, I performed normal forwarding logic. I searched the routing table using longest prefix match to find the appropriate outgoing route. If no route existed, I generated an ICMP destination network unreachable message and sent it back to the source. Before forwarding, I checked the TTL field. If the TTL was 1 or less, I dropped the packet and generated an ICMP time exceeded message, which is required for traceroute to work correctly. Otherwise, I decremented the TTL, recalculated the IP checksum, and proceeded. To forward the packet, I resolved the next hop's MAC address using the ARP cache. If an ARP entry was present, the packet was sent immediately after updating the Ethernet header. If no entry was found, I queued the packet in the ARP request queue and triggered ARP request handling. For ARP packets, I implemented both request and reply handling. The router responds to ARP requests only when the target IP matches one of its interfaces. For ARP replies, the router updates the ARP cache and forwards any queued packets waiting for that mapping.

ip_black_list Implementation

The `ip_black_list` function implements a simple firewall mechanism. Its purpose is to block both inbound and outbound packets belonging to a specific subnet. I implemented this by parsing the blacklisted network address (10.0.2.0) and its mask (255.255.255.0) and comparing them against both the source and destination IP addresses in the IP header. Since packet IP addresses are stored in network byte order, I converted them to host byte order before comparison. If either the source or destination IP belonged to the blacklisted subnet, the packet was dropped. In that case, the function prints a message in the required format indicating which IP was blocked. If no match was found, the function allows the packet to proceed. This function was designed to be simple, efficient, and independent of external libraries beyond standard C utilities.

sr_arpcache_handle_arpreq Implementation

The `sr_arpcache_handle_arpreq` function is responsible for managing outstanding ARP requests. It is called once per second to retransmit ARP requests and to handle failures. For each ARP request, I checked how much time had passed since the last transmission. If at least one second had elapsed and fewer than five requests had been sent, the router retransmits an ARP request for the target IP using the correct outgoing interface and a broadcast Ethernet address. If the request had already been sent five times without a reply, I treated it as a failure. In this case, the router sends an ICMP destination host unreachable message to the source of every packet waiting on that ARP request. After sending these ICMP messages, the ARP request entry is removed from the queue. This logic ensures correct ARP retry behavior, avoids infinite retries, and correctly notifies senders when a destination cannot be reached.

After implementation, I compiled the router using a clean build and tested it in the provided Mininet environment. The tests confirmed that the router behaves as specified and that all required protocols interact correctly.

Conclusion

Through this assignment, I implemented a functional IPv4 router that correctly handles packet forwarding, ARP resolution, ICMP messaging, and basic firewalling. The implementation closely follows protocol specifications and assignment requirements. This project strengthened my understanding of low-level networking, protocol interactions, and router control logic in a realistic environment.